

ACID-Garantien auf Databricks

Diese Seite erklärt, was ACID-Garantien sind und wie Databricks sie über Delta Lake umsetzt – obwohl die Daten letztlich in einem einfachen Cloud-Objektspeicher wie Amazon S3 liegen. Das Verständnis dieser Garantien ist wichtig, um zu wissen, warum man sich auf Databricks-Tabellen genauso verlassen kann wie auf eine klassische Datenbank.

Was bedeutet ACID?

ACID ist eine Abkürzung für vier Eigenschaften, die zusammen sicherstellen, dass Datenänderungen zuverlässig ablaufen: **Atomicity** (Atomarität), **Consistency** (Konsistenz), **Isolation** und **Durability** (Dauerhaftigkeit). Databricks verwendet standardmäßig Delta Lake als Speicherformat für alle Daten und liefert damit diese vier Garantien – ähnlich wie es klassische relationale Datenbanken tun. Wie genau die Garantien technisch umgesetzt werden, unterscheidet sich allerdings von System zu System.

Die vier ACID-Eigenschaften im Detail

Atomarität (Atomicity)

Atomarität bedeutet: Eine Transaktion gelingt entweder vollständig oder sie schlägt komplett fehl – ein "Teilerfolg" gibt es nicht. Databricks setzt das über ein sogenanntes **Transaktionsprotokoll** (Transaction Log) um. Während eine Schreiboperation läuft, werden Datendateien in das Tabellenverzeichnis geschrieben. Erst wenn der Vorgang vollständig abgeschlossen ist, wird ein neuer Eintrag im Transaktionsprotokoll festgeschrieben (committet) – inklusive der Pfade zu allen geschriebenen Dateien. Schlägt eine Transaktion mittendrin fehl, führt das nicht zu einer beschädigten Tabelle: Die bereits geschriebenen, aber nicht committeten Datendateien bleiben einfach ungenutzt liegen, bis der Befehl `VACUUM` sie irgendwann aufräumt.

Konsistenz (Consistency)

Databricks setzt Konsistenz über sogenannte **optimistische Nebenläufigkeitskontrolle** (Optimistic Concurrency Control) um. Das läuft in drei Schritten ab:

1. **Lesephase:** Das System liest die aktuellste Version der Tabelle, um herauszufinden, welche Dateien geändert werden müssen. Bei reinen Anfüge-Operationen (Append-only) entfällt dieser Schritt – stattdessen wird nur das Schema gegen die Metadaten im Transaktionsprotokoll geprüft.
2. **Schreibphase:** Die neuen Datendateien werden in das Tabellenverzeichnis geschrieben.
3. **Prüfung und Commit:** Das System prüft, ob seit der ursprünglichen Lesephase parallele Änderungen stattgefunden haben. Gibt es einen Konflikt, wird die Schreiboperation mit einer "Concurrent Modification Exception" abgelehnt. Gibt es keinen Konflikt, werden alle vorbereiteten Änderungen als neue, versionierte Momentaufnahme (Snapshot) übernommen.

Isolation

Databricks implementiert für Schreiboperationen eine sogenannte "write-serializable isolation" – eine besonders strenge Form der Isolation – kombiniert mit "Snapshot Isolation" für Lesezugriffe. Das bedeutet in der Praxis: Schreiboperationen erhalten stärkere Schutzgarantien als reine Lesezugriffe. Parallele Lesezugriffe werden dabei nur durch den Durchsatz des Metastores und der Cloud-Ressourcen begrenzt, nicht durch künstliche Sperren.

Dauerhaftigkeit (Durability)

Da Transaktionen entweder komplett gelingen oder komplett fehlschlagen und das Transaktionsprotokoll direkt neben den Datendateien liegt, erbt Databricks die Dauerhaftigkeits-Eigenschaften des zugrunde liegenden Cloud-Speichers (z. B. Amazon S3). Einmal erfolgreich geschriebene Daten gehen also nicht verloren.

Umfang von Transaktionen

Standardmäßig ist jede SQL-Anweisung ihre eigene atomare Transaktion gegen genau eine Tabelle. Es ist aber auch möglich, mehrere Anweisungen über mehrere Tabellen hinweg zu einer gemeinsamen Transaktion zu bündeln – mit der Syntax `BEGIN ATOMIC ... END; .` Voraussetzung dafür ist, dass für die beteiligten Tabellen sogenannte "Catalog Commits" aktiviert sind. Bei dieser Art von Transaktion gilt: Entweder gelingen alle Änderungen gemeinsam, oder es wird alles zurückgerollt (Rollback).

Optimistische statt klassische Sperren

Databricks setzt auf optimistische Nebenläufigkeitskontrolle statt auf klassische Sperrmechanismen (Locking). Das hat zwei Vorteile: Es schließt die Möglichkeit von sogenannten Deadlocks (gegenseitigen Blockaden) aus, und es erhöht den Durchsatz, weil parallele Lese- und Schreiboperationen sich nicht gegenseitig blockieren.

Mehrere Tabellen, mehrere Cluster

Über "Catalog Commits" unterstützt die Plattform auch Transaktionen über mehrere Tabellen hinweg – Änderungen an mehreren Tabellen lassen sich so als eine einzige atomare Operation koordinieren. Außerdem behandelt Databricks Schreibzugriffe von mehreren Clustern sicher: Delta Lake verhindert Datenbeschädigung, wenn mehrere Cluster gleichzeitig auf dieselbe Tabelle schreiben – auch wenn dabei einzelne Operationen miteinander in Konflikt geraten können, ohne dass die Daten dadurch beschädigt werden.

Technische Details

Jeder Schreibvorgang in Databricks erzeugt transaktionale Commits mit Metadaten-Dateien, die mit `_started_<id>` und `_committed_<id>` gekennzeichnet sind, zusätzlich zu den eigentlichen Datendateien. Das Aufräumen veralteter Commit-Metadaten übernimmt Databricks automatisch – hier ist kein manuelles Eingreifen nötig.

Wichtige Einschränkungen

- Andere Datenformate und angebundene externe Systeme bieten unter Umständen keine vergleichbaren Transaktionsgarantien für Lese- und Schreibzugriffe.
- Primär- und Fremdschlüssel-Beziehungen (Primary Key / Foreign Key) sind rein informativ – sie werden nicht technisch erzwungen.
- Delta Lake auf Amazon S3 hat einige zusätzliche Einschränkungen, die auf anderen Speichersystemen nicht gelten.

Fazit

Durch diese umfassende ACID-Umsetzung eignet sich Databricks auch für Analytics-Workloads, bei denen dieselbe Datenzuverlässigkeit gefordert ist wie bei einem klassischen Data Warehouse.

Quelle: <https://docs.databricks.com/aws/en/lakehouse/acid>

Stand: 2026-08-06